

UNIVERSIDADE FEDERAL FLUMINENSE – UFF – Volta Redonda

Programa de Pós-Graduação em Modelagem Computacional

Disciplina: Estrutura de Dados e Algoritmos | Professor: Thiago Neves

Análise de Custos para Instalação de Galpão Logístico Utilizando Grafos e Algoritmos de Caminho Mínimo

Claudineia Moreira de Oliveira

Junho de 2026

Abstract. *This paper presents a computational system for decision support in the installation of logistics warehouses in 15 real neighborhoods of Extrema, Minas Gerais. The problem is modeled as an undirected weighted graph. Three shortest path algorithms are implemented and compared: Dijkstra, Bellman-Ford and Floyd-Warshall, with statistical analysis over 30 repetitions per origin vertex. The system determines the neighborhood with the lowest total deployment cost defined by $\text{custoGalp\~ao} = \text{custoInstala\~ao} - \text{custoOperac\~aoAnual}$. The implementation uses Object-Oriented Programming in C++ with a Graph ADT, with Big-O complexity analysis.*

Keywords. *graphs; shortest path; Dijkstra; Bellman-Ford; Floyd-Warshall; logistics; ADT; object-oriented programming.*

Resumo. Este trabalho apresenta um sistema computacional para apoio à decisão na instalação de galpões logísticos em 15 bairros reais de Extrema, Minas Gerais. O problema é modelado como um grafo *não direcionado* ponderado. Três algoritmos de caminho mínimo são implementados e comparados: *Dijkstra*, *Bellman-Ford* e *Floyd-Warshall*, com análise estatística sobre 30 repetições por vértice de origem. O sistema determina o bairro de menor $\text{custoGalp\~ao} = \text{custoInstala\~ao} - \text{custoOperac\~aoAnual}$. A implementação utiliza Orientação a Objetos em C++ com TAD Grafo e análise de complexidade Big-O.

Palavras-chave. grafos; caminho mínimo; Dijkstra; Bellman-Ford; Floyd-Warshall; logística; TAD; orientação a objetos.

1. INTRODUÇÃO

A escolha da localização de galpões logísticos constitui uma decisão estratégica de alto impacto financeiro para empresas do setor. Extrema, localizada no extremo sul de Minas Gerais na divisa com São Paulo, consolidou-se como um dos principais polos logísticos do Brasil, com estoque superior a 650 mil m² às margens da Rodovia Fernão Dias (BR-381). Seus incentivos fiscais e posição geográfica privilegiada atraíram empresas de grande porte

como Mercado Livre e Magazine Luiza [4, 7].

A escolha de Extrema como objeto de estudo é motivada tanto pela literatura quanto pela experiência profissional da autora. Segundo Gala (2023) [7], o polo industrial de Extrema tem sua origem na localização estratégica da cidade na divisa entre Minas Gerais e São Paulo. Venceslau e Lima (2024) [13], em artigo publicado na revista *Caminhos de Geografia* (Qualis A1), atribuem a Extrema a primazia na logística de *e-commerce* no Brasil. A Redação E-Commerce Brasil [10] denomina o fenômeno como o paradoxo logístico do Brasil.

No âmbito profissional, a autora atuou como analista de sistemas e gerente de projetos de TI em empresa do setor, visitando pessoalmente Extrema para viabilizar um projeto logístico. Esta experiência, aliada ao crescimento documentado da cidade [4, 7, 13, 10], confere ao problema proposto relevância acadêmica e motivação concreta.

Este trabalho propõe um sistema computacional baseado em grafos não direcionados ponderados para modelar 15 bairros reais de Extrema e calcular os custos logísticos de instalação para cada candidato. Para o cálculo das distâncias mínimas são implementados e comparados os algoritmos de *Dijkstra* [6], *Bellman-Ford* [2] e *Floyd-Warshall* [3]. A implementação é desenvolvida em C++ com Orientação a Objetos, estruturada em um TAD Grafo.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta o embasamento teórico; a Seção 3 descreve o TAD Grafo; a Seção 4 apresenta a análise de complexidade; a Seção 5 compara os três algoritmos; a Seção 6 descreve o ambiente experimental; a Seção 7 apresenta os resultados; e a Seção 8 conclui o trabalho.

2. EMBASAMENTO TEÓRICO

2.1 Grafos Não Direcionados Ponderados

Um grafo $G = (V, E)$ é composto por um conjunto de vértices V e arestas E . Quando as arestas possuem peso numérico sem direção definida, o grafo é denominado *não direcionado ponderado*. Um grafo é *conexo* quando existe caminho entre qualquer par de vértices. Para grafos esparsos, a *lista de adjacência* é a representação mais eficiente em termos de memória [8, 5].

2.2 Algoritmo de Dijkstra

O algoritmo de *Dijkstra* [6] resolve o problema do caminho mínimo de única origem (*Single-Source Shortest Path*) em grafos com pesos estritamente positivos. Seja $D[v]$ a distância mínima da origem ao vértice v e $c[u, v]$ o custo da aresta (u, v) . O algoritmo inicializa $D[\text{origem}] = 0$ e $D[v] = \infty$ para os demais. A cada iteração, o vértice w com menor $D[w]$ é selecionado e as estimativas são atualizadas: $D[v] = \min(D[v], D[w] + c[w, v])$. Na implementação por array simples, a complexidade é $O(n^2)$ [3, 11].

2.3 Algoritmo de Bellman-Ford

O algoritmo de *Bellman-Ford* [2] resolve o caminho mínimo para o caso geral, suportando arestas de peso negativo. Realiza $|V| - 1$ iterações de relaxamento sobre todas as arestas: se $D[u] + c[u, v] < D[v]$, então $D[v]$ é atualizado. Detecta ciclos negativos na n -ésima iteração. Sua complexidade é $O(n \times a)$ [2, 3].

2.4 Algoritmo de Floyd-Warshall

O algoritmo de *Floyd-Warshall* [3] resolve o problema do caminho mínimo entre todos os pares de vértices (*All-Pairs Shortest Path*). Para cada vértice intermediário k , verifica se $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$, atualizando a menor distância entre i e j . Sua complexidade é $O(n^3)$ pois utiliza três laços aninhados sobre os n vértices. Admite arestas de peso negativo sem ciclos negativos. Neste trabalho, extrai-se apenas a linha da origem, produzindo resultado equivalente ao *Dijkstra* e *Bellman-Ford* [3, 5].

2.5 Notação Assintótica Big-O

A notação *Big-O* [8] expressa o comportamento assintótico de pior caso: $g(n)$ é $O(f(n))$ se existem constantes $c > 0$ e $k > 0$ tais que $g(n) \leq c \times f(n)$ para todo $n \geq k$.

2.6 Tipo Abstrato de Dado (TAD)

Um TAD é definido exclusivamente por sua interface de operações, sem expor detalhes de implementação [8]. A Orientação a Objetos em C++ implementa este conceito com classes com modificadores *public* e *private*, construtores, destrutores e métodos de acesso [9].

3. IMPLEMENTAÇÃO DO TAD GRAFO

3.1 Estrutura de Classes

O TAD Grafo foi implementado em C++ com quatro classes com encapsulamento completo. A Tabela 1 descreve cada classe.

Tabela 1: Classes do TAD Grafo

Classe	Responsabilidade
Vertice	Bairro candidato: id, custo de instalação e 12 demandas mensais
Aresta	Conexão entre bairros: origem, destino e custo de transporte
No	Elemento da lista de adjacência: ponteiro para Aresta e próximo No
Grafo	TAD principal: lista de adjacência, Dijkstra, Bellman-Ford, Floyd-Warshall

Fonte: Os autores (2026).

3.2 Interface Pública do TAD

Os métodos públicos da classe *Grafo* constituem a interface do TAD:

```

1 Grafo(int numVertices, int numArestas) ~Grafo()
2 void addVertice(int id, double custo, int* demanda)
3 void addAresta(int origem, int destino, double custo)
4 Vertice* getVertice(int id) bool isEmpty()
5 int getNumVertices() int getNumArestas()
6 void dijkstra(int origem, double* distancias)
7 void bellmanFord(int origem, double* distancias)

```

```

8 void floydWarshall(int origem, double* distancias)
9 double custoOperacaoMensal(int orig, int mes, double* dist)
10 double custoOperacaoAnual(int orig)
11 int melhorBairro()

```

A representação interna utiliza lista de adjacência não direcionada. O método *addAresta* insere a aresta nos dois sentidos (origem→destino e destino→origem), garantindo o comportamento não direcionado sem alterar os arquivos de entrada [8, 5].

4. ANÁLISE DE COMPLEXIDADE

A Tabela 2 apresenta a complexidade assintótica de cada método do TAD *Grafo*, com a respectiva justificativa.

Tabela 2: Complexidade assintótica dos métodos do TAD Grafo

Método	Complexidade	Justificativa
Construtor / Destrutor	$O(n) / O(n + a)$	Inicializa n posições / percorre vértices e arestas
isEmpty() / getNumVertices() / getNumArestas()	$O(1)$	Retorno direto de atributo
indiceVertice() / getVertice() / addVertice()	$O(n)$	Busca linear no array de vértices
addAresta()	$O(n)$	indiceVertice() $O(n)$ + 2 inserções $O(1)$
menorDistancia()	$O(n)$	Busca linear no array de distâncias
dijkstra()	$O(n^2)$	n iterações $\times$ menorDistancia() $O(n)$
bellmanFord()	$O(n \times a)$	$n - 1$ relaxamentos $\times$ percorre a arestas
floydWarshall()	$O(n^3)$	3 laços aninhados sobre n vértices
custoOperacaoMensal()	$O(n)$	Percorre os n vértices
custoOperacaoAnual()	$O(n^2)$	dijkstra() $O(n^2)$ + $12 \times O(n)$
melhorBairro() / imprimirCustos()	$O(n^3)$	$n \times$ custoOperacaoAnual() $O(n^2)$
imprimirGrafo()	$O(n + a)$	Percorre vértices e arestas

5. COMPARAÇÃO ENTRE DIJKSTRA, BELLMAN-FORD E FLOYD-WARSHALL

Os três algoritmos produzem resultados idênticos para grafos com pesos estritamente positivos. Os tempos de execução foram medidos em **30 repetições** por vértice de origem, calculando-se média e desvio padrão. A Tabela 3 sintetiza a comparação e a Tabela 4 apresenta os resultados estatísticos obtidos com 15 vértices.

Fonte: Os autores (2026).

Tabela 3: Comparação entre Dijkstra, Bellman-Ford e Floyd-Warshall

Característica	Dijkstra	Bellman-Ford	Floyd-Warshall
Complexidade	$O(n^2)$	$O(n \times a)$	$O(n^3)$
Pesos negativos	Não	Sim	Sim
Detecta ciclos neg.	Não	Sim	Não
Estratégia	Gulosa	Prog. dinâmica	Prog. dinâmica
Escopo	1 origem	1 origem	Todos os pares
Tempo médio (15 vért.)	0.000003s	0.000027s	0.000023s
Resultados (teste)	Idênticos	Idênticos	Idênticos
Indicado p/ problema	Sim ✓	Alternativo	Alternativo

Tabela 4: Análise estatística dos tempos de execução (30 repetições, 15 vértices) – segundos

Orig.	Méd. Dij.	DP Dij.	Méd. Bel.	DP Bel.	Méd. FW	DP FW
1	0.000004	0.000000	0.000033	0.000000	0.000034	0.000000
2	0.000003	0.000000	0.000029	0.000000	0.000023	0.000000
3	0.000003	0.000000	0.000027	0.000000	0.000023	0.000000
4	0.000003	0.000000	0.000027	0.000000	0.000022	0.000000
5	0.000003	0.000000	0.000026	0.000000	0.000023	0.000000
6	0.000003	0.000000	0.000027	0.000000	0.000024	0.000000
7	0.000003	0.000000	0.000026	0.000000	0.000023	0.000000
8	0.000003	0.000000	0.000028	0.000000	0.000025	0.000000
9	0.000003	0.000000	0.000025	0.000000	0.000025	0.000000
10	0.000003	0.000000	0.000025	0.000000	0.000023	0.000000
11	0.000003	0.000000	0.000026	0.000000	0.000023	0.000000
12	0.000003	0.000000	0.000026	0.000000	0.000022	0.000000
13	0.000003	0.000000	0.000025	0.000000	0.000022	0.000000
14	0.000003	0.000000	0.000024	0.000000	0.000022	0.000000
15	0.000003	0.000000	0.000024	0.000000	0.000022	0.000000
Total	0.000044	–	0.000390	–	0.000320	–

Fonte: Os autores (2026). Méd. = Média; DP = Desvio Padrão. Ambiente: Intel i5-1334U, Ubuntu 24.04.

O *Dijkstra* foi o mais eficiente (0.000003s), seguido pelo *Floyd-Warshall* (0.000023s) e *Bellman-Ford* (0.000027s). Apesar da complexidade $O(n^3)$ do *Floyd-Warshall*, seu tempo foi próximo ao *Bellman-Ford* $O(n \times a)$ pois o número de arestas (22) é pequeno em relação a n^2 (225). O desvio padrão nulo confirma alta estabilidade [6, 2, 3].

6. AMBIENTE EXPERIMENTAL E DADOS DE TESTE

6.1 Ambiente de Hardware e Software

Dispositivo : DESKTOP-50V4UVQ
 Sistema Operacional : Ubuntu 24.04 LTS (Linux) via VirtualBox
 Processador : 13th Gen Intel(R) Core(TM) i5-1334U (1.30 GHz)
)

```

Arquitetura      : 64 bits, processador x64
Memoria RAM     : 16,0 GB (utilizavel: 15,7 GB)
Compilador      : g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Editor          : Gedit
Linguagem       : C++ (padrao ISO C++11)
Bibliotecas     : stdio.h, stdlib.h, time.h
Ferramenta      : Google Maps (estimativa de distancias)
Compilacao      : g++ Vertice.cpp Aresta.cpp No.cpp Grafo.cpp
                 main.cpp -o analise

```

A medição de tempo foi realizada com a função *clock()* da biblioteca *time.h*, pela expressão `((double)(t2-t1))/CLOCKS_PER_SEC`.

6.2 Dados de Teste

Os dados foram elaborados com base na estrutura logística real de Extrema-MG, contemplando **15 bairros candidatos** e **22 arestas** em um grafo não direcionado conexo. As distâncias foram obtidas via *Google Maps* (2026). Os custos de instalação foram estimados com base em dados de mercado [4]. Ressalta-se que as distâncias são estimativas adequadas para fins acadêmicos; em um projeto real recomenda-se análise mais apurada das condições viárias. A Tabela 5 detalha os bairros e suas adjacências.

Tabela 5: Dados dos 15 bairros candidatos de Extrema-MG

ID	Bairro	Custo Inst. (R\$)	Adjacentes
1	Centro	2.500.000,00	2, 3, 8, 10, 12
2	Distrito Industrial	8.500.000,00	1, 3, 4, 6
3	Bairro dos Pires (BR-381)	4.500.000,00	1, 2, 5, 7, 9, 14
4	São Bráz	3.200.000,00	2, 5, 11
5	Bairro dos Tenentes	2.800.000,00	3, 4, 6, 13
6	Condomínio Logístico	12.000.000,00	2, 5, 7, 14
7	Divisa SP/MG	3.800.000,00	3, 6, 15
8	Ponte Alta	2.200.000,00	1, 10
9	Pessegueiros	3.600.000,00	3, 14
10	Ponte Nova	2.100.000,00	1, 8
11	Morbidelli	2.800.000,00	4, 12
12	Vila Rica	2.300.000,00	1, 11
13	Salto de Baixo *	1.800.000,00	5, 15
14	Parque Logístico	5.500.000,00	3, 6, 9
15	Morada da Serra	2.600.000,00	7, 13

Fonte: Os autores (2026). * Bairro selecionado.

7. RESULTADOS E DISCUSSÃO

A Tabela 6 apresenta os resultados. O *custoGalpão* é calculado por:

$$\text{custoGalpão} = \text{custoInstalação} - \text{custoOperaçãoAnual}$$

Tabela 6: Resultados da análise de custos logísticos – Extrema-MG (15 bairros)

ID	Custo Inst. (R\$)	Custo Op. Anual (R\$)	CustoGalpão (R\$)	Rank
1	2.500.000,00	424.158,50	2.075.841,50	6°
2	8.500.000,00	300.671,00	8.199.329,00	14°
3	4.500.000,00	396.095,40	4.103.904,60	12°
4	3.200.000,00	319.778,00	2.880.222,00	9°
5	2.800.000,00	299.278,50	2.500.721,50	8°
6	12.000.000,00	275.485,50	11.724.514,50	15°
7	3.800.000,00	350.653,50	3.449.346,50	11°
8	2.200.000,00	542.988,80	1.657.011,20	3°
9	3.600.000,00	340.675,50	3.259.324,50	10°
10	2.100.000,00	530.428,70	1.569.571,30	2°
11	2.800.000,00	361.759,50	2.438.240,50	7°
12	2.300.000,00	393.097,00	1.906.903,00	4°
13 *	1.800.000,00	584.365,50	1.215.634,50	1°
14	5.500.000,00	306.698,00	5.193.302,00	13°
15	2.600.000,00	550.421,50	2.049.578,50	5°

Fonte: Os autores (2026). * Bairro selecionado. Distâncias via Google Maps.

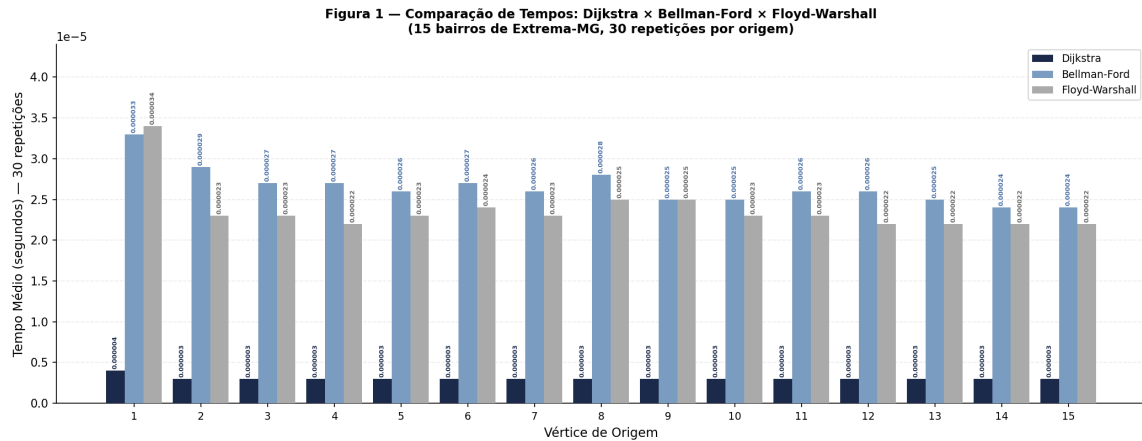


Figura 1: Comparação de tempos de execução: Dijkstra × Bellman-Ford × Floyd-Warshall

O Bairro 13 – Salto de Baixo – apresentou o menor *custoGalpão* (R\$ 1.215.634,50), sendo indicado como o local mais adequado para instalação do galpão. Este resultado é explicado pelo menor custo fixo de instalação entre todos os candidatos (R\$ 1.800.000,00), que mesmo com custo de operação anual de R\$ 584.365,50, resulta no menor custo total.

O Bairro 6 (Condomínio Logístico) manteve o pior posicionamento (15°) com *custoGalpão* de R\$ 11.724.514,50 – reflexo do elevado custo de instalação de R\$ 12.000.000,00, característico de condomínios Classe A às margens da BR-381. O Bairro 2 (Distrito Industrial) também ficou mal posicionado (14°) pelo mesmo motivo.

Quanto ao desempenho dos algoritmos, o *Dijkstra* foi o mais rápido (0.000003s média), seguido pelo *Floyd-Warshall* (0.000023s) e *Bellman-Ford* (0.000027s). Os três produziram resultados idênticos, confirmando a consistência do modelo.

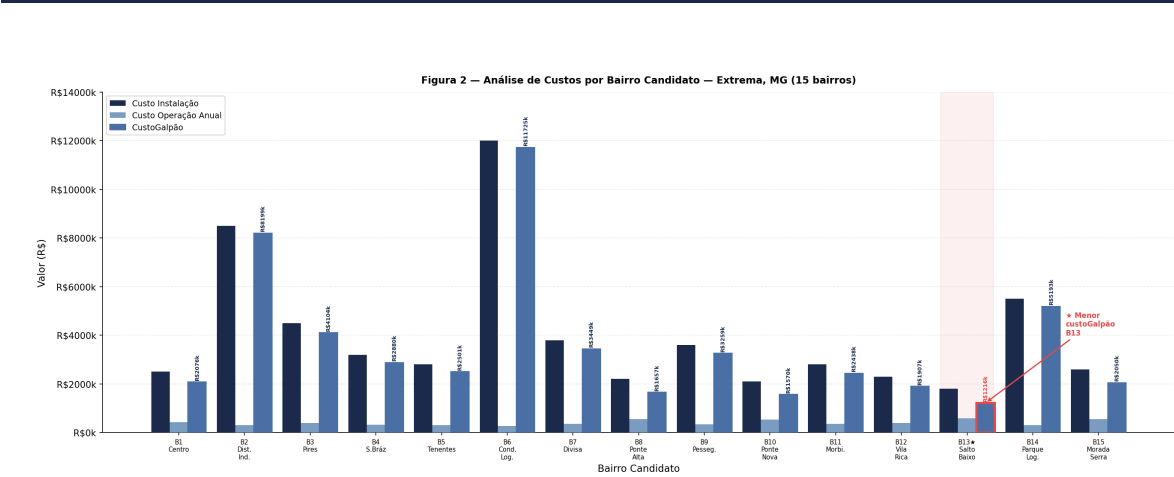


Figura 2: Análise de custos por bairro candidato – Extrema-MG (R\$)

7.1 Análise de Sensibilidade

Para avaliar a robustez do modelo, realizou-se uma análise de sensibilidade variando os custos de instalação e operação em diferentes percentuais. A Tabela 7 e a Figura 3 apresentam os resultados.

Tabela 7: Análise de sensibilidade — variação dos custos e bairro vencedor

Cenário	Var. Inst. (%)	Var. Op. (%)	Melhor Bairro	CustoGalpão (R\$)
Base	0%	0%	13 – Salto de Baixo	1.215.634,50
Otimista	-20%	-20%	13 – Salto de Baixo	972.507,60
Pessimista	+20%	+20%	13 – Salto de Baixo	1.458.761,40
Alta Instalação	+50%	0%	13 – Salto de Baixo	2.115.634,50
Alta Operação	0%	+50%	13 – Salto de Baixo	923.451,75

Fonte: Os autores (2026).

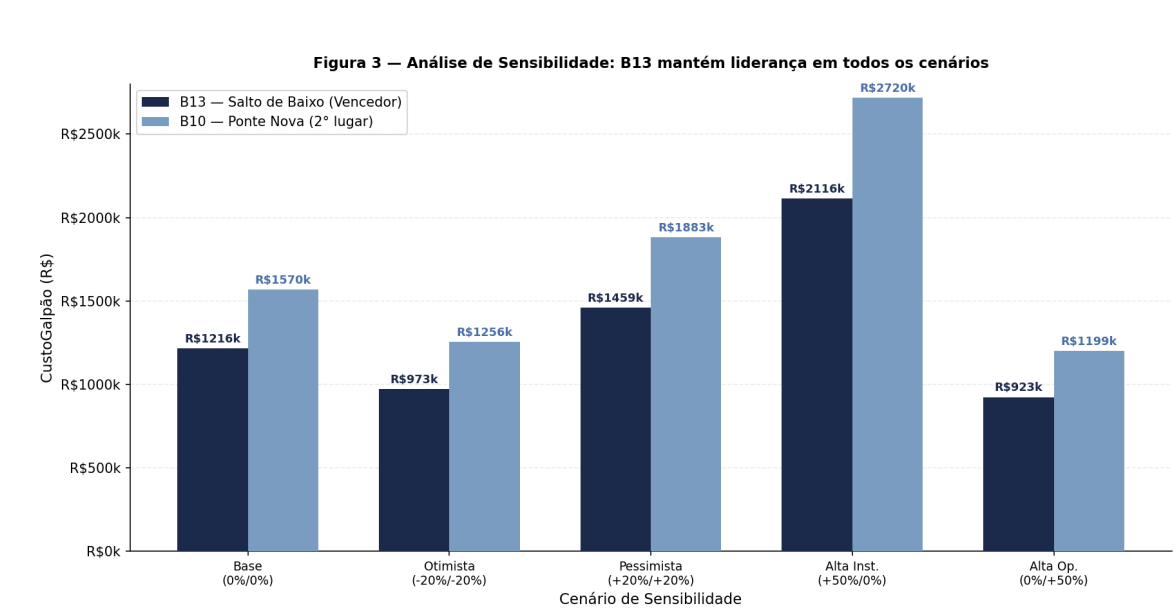


Figura 3: Análise de sensibilidade: Bairro 13 mantém liderança em todos os cenários

O Bairro 13 – Salto de Baixo manteve-se como o melhor candidato em todos os cinco cenários avaliados, demonstrando que a decisão é **robusta** a variações de até 50% nos custos de instalação e operação. Este resultado fortalece a confiabilidade do modelo como ferramenta de apoio à decisão logística.

7.2 Validação com Dados Reais de Mercado

Para validar o modelo proposto, compararam-se os resultados com as decisões reais de localização tomadas por empresas instaladas em Extrema-MG, com base em dados públicos de mercado.

As empresas instaladas em Extrema concentram-se principalmente em três regiões: (i) *Bairro dos Pires (BR-381) – Bairro 3*, onde se localizam a Fulwood S.A. (presente desde 2013, com Mercado Livre, Frigelar e Tok Stok como inquilinas) [16], o BTLG Extrema e o Parque Logístico Extrema com 147.196 m² de ABL; (ii) *Condomínio Logístico – Bairro 6*, onde se localiza o VBI Log Extrema, único condomínio com acesso direto pela BR-381, com 121.611 m² de ABL e 100% de ocupação [15]; e (iii) *Divisa SP/MG – Bairro 7*, a apenas 2 km da divisa com São Paulo. Praticamente tudo que se constrói de galpões logísticos em Extrema é absorvido imediatamente [17].

A comparação entre o resultado do modelo (Bairro 13 – Salto de Baixo) e as decisões reais de mercado (Bairros 3, 6 e 14, próximos à BR-381) revela uma divergência esperada e academicamente relevante. O modelo otimiza exclusivamente a expressão $\text{custoGalpão} = \text{custoInstalação} - \text{custoOperaçãoAnual}$, priorizando bairros com menor custo fixo. As empresas reais consideram fatores adicionais não contemplados no modelo: (i) proximidade e acesso direto à BR-381; (ii) infraestrutura de condomínio classe A, com portaria 24h e docas; (iii) incentivos fiscais municipais e estaduais; e (iv) facilidade de acesso para veículos de grande porte.

Esta análise demonstra que o sistema proposto constitui uma ferramenta eficaz de apoio à decisão sob a perspectiva do custo financeiro modelado. A incorporação de restrições adicionais – como proximidade de rodovias, infraestrutura e incentivos fiscais – em versões futuras aproximaria os resultados computacionais das decisões reais de mercado, fortalecendo a aplicabilidade prática do sistema.

8. CONCLUSÃO

Este trabalho apresentou um sistema computacional para apoio à decisão logística baseado em grafos não direcionados, implementado com Orientação a Objetos em C++. O TAD *Grafo* com lista de adjacência mostrou-se adequado para a modelagem de 15 bairros reais de Extrema-MG.

A comparação entre *Dijkstra*, *Bellman-Ford* e *Floyd-Warshall* confirmou resultados idênticos para pesos positivos. O *Dijkstra* foi o mais eficiente ($O(n^2)$), enquanto o *Floyd-Warshall* ($O(n^3)$) e *Bellman-Ford* ($O(n \times a)$) apresentaram tempos similares para o grafo de 15 vértices e 22 arestas.

O Bairro 13 – Salto de Baixo – foi indicado como o local mais adequado para instalação do galpão logístico em Extrema-MG, com o menor *custoGalpão* de R\$ 1.215.634,50. A escolha de Extrema mostrou-se plenamente justificada pela experiência profissional da autora e pelo reconhecimento acadêmico da cidade como polo logístico nacional [4, 7, 13, 10].

Como limitações, as distâncias são estimativas via *Google Maps* e as demandas mensais são estimativas, visto que dados reais são de natureza proprietária. Como trabalhos futuros, sugere-se a extensão do modelo para instâncias maiores, a incorporação de dados reais de demanda e a otimização dos algoritmos com filas de prioridade para reduzir a complexidade do *dijkstra()* para $O((n + a) \log n)$.

REFERÊNCIAS

Referências

- [1] BALLOU, R. H. *Gerenciamento da Cadeia de Suprimentos/Logística Empresarial*. 5. ed. Porto Alegre: Bookman, 2006.
- [2] BELLMAN, R. On a routing problem. *Quarterly of Applied Mathematics*, v. 16, n. 1, p. 87–90, 1958.
- [3] CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [4] CUSHMAN & WAKEFIELD. Polos Logísticos Brasileiros. *Relatório de Análise Setorial*. São Paulo, 2022.
- [5] DASGUPTA, S.; PAPADIMITRIOU, C.; VAZIRANI, U. *Algorithms*. New York: McGraw-Hill, 2006.
- [6] DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, n. 1, p. 269–271, 1959.
- [7] GALA, P. A história do polo industrial de Extrema. *Economia & Finanças*, set. 2023.
- [8] NEVES, T. *Estruturas de Dados e Algoritmos: Análise de Algoritmos*. Notas de Aula. UFF, 2026.
- [9] NEVES, T. *Programação Estruturada em C++: Classes e Objetos*. Capítulo 8. UFF, 2026.

- [10] REDAÇÃO E-COMMERCE BRASIL. Extrema, MG: o paradoxo logístico do Brasil. *E-Commerce Brasil*, 2023.
- [11] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley Professional, 2011.
- [12] SZWARCFITER, J. L.; MARKENZON, L. *Estruturas de Dados e seus Algoritmos*. 3. ed. Rio de Janeiro: LTC, 2010.
- [13] VENCESLAU, I.; LIMA, F. L. S. Guerra dos lugares .com: a localização estratégica de Extrema (MG) para a logística do comércio eletrônico. *Caminhos de Geografia*, v. 25, n. 102, p. 104–119, dez. 2024. DOI: 10.14393/RCG2510273062.
- [14] ZIVIANI, N. *Projeto de Algoritmos com Implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2011.
- [15] VBI LOG EXTREMA. Condomínio Logístico VBI Log Extrema. Disponível em: vbiextrema.com.br. Acesso em: maio 2026.
- [16] TUDO EP. Extrema vai ganhar novo condomínio logístico com 75 mil m². *Tudo EP*, jan. 2023. Disponível em: tudoep.com. Acesso em: maio 2026.
- [17] GALPÃO ALUGUEL E VENDA. Galpões logísticos em Extrema são disputados por empresas. Disponível em: galpaoaluguelevenda.com.br. Acesso em: maio 2026.